Oralable MAM Platform Documentation

Firmware, iOS BLE, GitHub Repositories & Algorithm Integration

Oralable / johnacogan67

June 7, 2026

Contents

1 Part I — Upload Index

```
ORALABLE DOCUMENTATION PACK — UPLOAD INDEX
Generated: 2026-06-07

This folder contains text files for uploading to external systems (partners,
regulatory, grant applications, architecture reviews).

FILES
-----
01_FIRMWARE_ARCHITECTURE.txt
  nRF52832 firmware (oralable_nrf): GATT layout, worn-gating, BLE policy,
  state machine, build/flash, version history. Current: 1.0.36-nrfconnect.

02_IOS_BLE_STREAMING_SUMMARY.txt
  iOS app BLE stack (oralable_swift + OralableCore): connection flow,
  parsing pipeline, coordinators, background worker, gaps for 8+ hour sessions.

03_GITHUB_REPOS_OVERVIEW.txt
  All four GitHub repos: URLs, commits, README copy-paste, directory structure,
  cross-repo relationships.

04_ALGORITHM_AND_SYSTEM_INTEGRATION.txt
  Python <-> Swift algorithm architecture, signal processing rules, ML path,
  validation strategy, known gaps.

GITHUB REPOSITORIES
-------------------
https://github.com/johnacogan67/cursor_oralable   (Python research)
https://github.com/johnacogan67/oralable_nrf       (nRF firmware)
https://github.com/johnacogan67/oralable_swift     (iOS app)
https://github.com/johnacogan67/OralableCore         (shared Swift package)

LOCAL PATHS
-----------
/Users/johnacogan67/work/cursor_oralable
/Users/johnacogan67/work/oralable_nrf
/Users/johnacogan67/work/oralable_swift
/Users/johnacogan67/work/OralableCore
```

2 Part II — Firmware Architecture

```
================================================================================
ORALABLE MAM — FIRMWARE ARCHITECTURE (oralable_nrf)
Generated: 2026-06-07
Current version: 1.0.36-nrfconnect (app/VERSION)
Target board: pcb00003 (nRF52832, revision 6)
================================================================================

1. REPOSITORY
-------------
URL:     https://github.com/johnacogan67/oralable_nrf
Branch:  known-good-battery-ble (local)
Commits: 4 (tracked in git; workspace includes full NCS/Zephyr tree)
App dir: oralable_nrf/app/
Build:   west build -b pcb00003 -d build_pcb00003 app -- -DBOARD_ROOT=<repo>
Flash:   ./scripts/flash_and_rtt.sh --build-dir build_pcb00003 --snr <J-Link SN>

2. HIGH-LEVEL ARCHITECTURE
--------------------------

+-------------------------------------------------------------------------+
|  main.c                                                                 |
|  - Device state machine (worn / charging / init)                        |
|  - Die-temp worn detection (25.5°C on, 24.5°C off hysteresis)           |
|  - Calls tgm_service_set_device_worn() on state transitions             |
|  - Temperature sampling work queue (for worn + status)                  |
`-------------------------------+-----------------------------------------+
                                |
        +-----------------------+-----------------------+
        v                       v                       v
+---------------+      +-----------------+     +-----------------+
|  ble.c        |      |  tgm_service.c  |     |  Sensor drivers |
|  - Peripheral |<---->|  GATT service   |     |  ppg.c (MAXM86161)|
|  - Adv/conn   |      |  10 chars       |     |  acc.c (LIS2DTW12)|
|  - Disconnect |      |  Notify paths   |     |  battery.c (ADC)  |
|    logging    |      |  Worn gating    |     |  sensor_i2c.c     |
`---------------+      `-----------------+     `-----------------+

3. SOURCE FILES (app/src/)
--------------------------
main.c          Application entry, worn/charging state, temp work
ble.c / ble.h   Bluetooth peripheral, connection lifecycle
tgm_service.c/h Custom GATT service (TGM / Oralable protocol)
ppg.c / ppg.h   MAXM86161 PPG driver, 50 Hz sampling, frame batching
acc.c / acc.h   LIS2DTW12 accelerometer, 50 Hz sampling
battery.c/h     Battery voltage ADC, mV reporting
sensor_i2c.c/h  Shared I2C mutex for sensor bus
early_rtt.c     SEGGER RTT early logging

4. GATT SERVICE LAYOUT (UUID base 3A0FF000-98C4-46B2-94AF-1AEE0FD4C48E)
------------------------------------------------------------------------
Char UUID suffix | Name              | Direction | Payload
-----------------|-------------------|-----------|---------------------------------
001              | PPG               | Notify    | frame_counter + N×(R4+IR4+G4) bytes
002              | ACC               | Notify    | frame_counter + N×(X2+Y2+Z2) bytes
```

```
003              | Temp              | Notify    | frame_counter + centi-°C int16
004              | Battery           | Notify    | int32 millivolts
005              | Device ID         | Read      | uint64
006           | Firmware        | Read     | version string (e.g. "1.0.36-nrfconnect")
007              | PPG reg read      | Notify    | register value
008              | PPG reg write     | Write     | 2 bytes: reg addr + value
009           | Status          | Notify   | charging, worn, device_state, battery_pct

5. DATA FORMATS
---------------
PPG (CONFIG_PPG_SAMPLES_PER_FRAME=20):
  - 50 Hz effective rate; 20 samples/frame -> ~0.4 s per BLE packet (~244 bytes)
  - Per sample: Red uint32, IR uint32, Green uint32 (12 bytes)
  - Frame header: uint32 frame_counter

ACC (CONFIG_ACC_SAMPLES_PER_FRAME=25):
  - 50 Hz; 25 samples/frame -> ~0.5 s per packet (~154 bytes)
  - Per sample: int16 X, Y, Z (6 bytes)
  - Frame header: uint32 frame_counter

Temperature:
  - 8 bytes: frame_counter (4) + centi-degrees int16 (2)
  - CONFIG_TEMPERATURE_MEASUREMENT_INTERVAL=1 s

Battery:
  - int32 millivolts (4 bytes)
  - CONFIG_BATTERY_MEASUREMENT_INTERVAL=300 s (5 min) for scheduled reads
  - Link keepalive also sends battery every 5 s when battery CCC enabled

Status (009):
  - byte0: charging (0/1)
  - byte1: worn (0/1) — body temp detected
  - byte2: device_state enum (0-3)
  - byte3: battery_pct (0-100)

6. DEVICE STATE MACHINE (main.c)
--------------------------------
enum device_state_t:
  DEVICE_STATE_NOT_WORN_NOT_CHARGING
  DEVICE_STATE_NOT_WORN_CHARGING
  DEVICE_STATE_WORN
  DEVICE_STATE_INIT

Worn detection:
  - nRF die temperature sensor
  - Worn threshold:  >= 25.50°C (2550 centidegrees)
  - Not-worn threshold: < 24.50°C (2450 centidegrees)
  - On transition: tgm_service_set_device_worn(bool)

7. BLE CONNECTION POLICY (prj.conf + tgm_service.c)
---------------------------------------------------
Connection params (peripheral preferred):
  - Min interval: 12 × 1.25 ms = 15 ms
  - Max interval: 36 × 1.25 ms = 45 ms
  - Latency: 0
  - Supervision timeout: 3200 × 10 ms = 32 s (NCS max for nRF52832)
```

```
Buffer hardening (1.0.35+):
  - CONFIG_BT_L2CAP_TX_BUF_COUNT=12
  - CONFIG_BT_ATT_TX_COUNT=12
  - gatt_notify_checked(): 80 ms backoff on -ENOMEM

Link keepalive (1.0.34+):
  - k_work_delayable every 5 s while connected
  - Sends battery notify if battery CCC enabled
  - Re-armed on every CCC enable and after each fire

8. WORN-GATED STREAMING (1.0.36)
--------------------------------
tgm_service_set_device_worn(bool worn):

OFF-BODY (worn=false):
  - ppg_stop(), acc_stop(), ppg_set_notify_poll(false)
  - Block PPG/ACC/temp notify sends even if CCC still enabled
  - Temp still sampled internally for worn detection
  - Status notify on worn/charging change (if 009 CCC on)
  - ~50× less BLE traffic vs full streaming

ON-BODY (worn=true):
  - If PPG/ACC CCC still enabled: deferred ppg_start/acc_start via k_work
  - Full 50 Hz PPG + ACC streaming resumes without re-enabling CCC

CCC handlers:
  - PPG CCC on -> link_keepalive_arm + ppg_start_work (only starts if worn)
  - PPG CCC off -> ppg_stop
  - ACC CCC on/off -> same pattern
  - Temp/battery/status CCC -> flag only; data gated by worn where applicable

9. SENSOR HARDWARE
------------------
PPG:  Maxim MAXM86161 (I2C) — Red, IR, Green LEDs
ACC:  ST LIS2DTW12 (I2C) — 3-axis accelerometer
MCU:  nRF52832 on pcb00003
Battery: ADC measurement, charging detect via GPIO (chrsts)

10. MEMORY (typical build_pcb00003)
-----------------------------------
FLASH: ~194 KB / 512 KB (37%)
RAM:   ~42 KB / 64 KB (64%)

11. FIRMWARE VERSION HISTORY (recent)
-------------------------------------
1.0.32  Supervision/battery fixes, TGM UUID in scan response
1.0.33  Deferred PPG/ACC start via k_work, I2C mutex, static notify buffers
1.0.34  32s supervision, PPG 500ms poll fallback, 5s link keepalive
1.0.35  L2CAP/ATT buffer increase, notify backpressure, keepalive reschedule fix
1.0.36  Worn-gated streaming (off-body minimal BLE, on-body full rate)

12. BUILD / FLASH COMMANDS
--------------------------
cd /Users/johnacogan67/work/oralable_nrf
west build -b pcb00003 -d build_pcb00003 app -- -DBOARD_ROOT=/Users/johnacogan67/work/oralable_nrf
```

```
./scripts/flash_and_rtt.sh --build-dir build_pcb00003 --snr 1050090445

Note: J-Link error -256 during flash is benign noise; erase/program/verify still succeeds.

13. VALIDATION (nRF Connect log 11, firmware 1.0.36)
-----------------------------------------------------
Device off-body, all 7 CCCs enabled:
  - PPG=0, ACC=0, TEMP=0 packets for ~130 s after last CCC
  - Link stayed up ~3 min with minimal traffic
  - Confirms worn-gating works as designed

Pending validation:
  - On-body resume without re-enabling CCC
  - Status 009 notify on worn transitions
  - iOS 8+ hour session with adaptive download

================================================================================
END FIRMWARE ARCHITECTURE
================================================================================
```

3 Part III — iOS BLE Streaming Summary

```
================================================================================
ORALABLE MAM — iOS BLE STREAMING LOGIC SUMMARY
Repositories: oralable_swift (app) + OralableCore (shared package)
Generated: 2026-06-07
================================================================================

1. REPOSITORIES
---------------
oralable_swift:  https://github.com/johnacogan67/oralable_swift  (398 commits, 433 files)
OralableCore:    https://github.com/johnacogan67/OralableCore    (60 commits, 127 files)

OralableCore is a Swift Package (iOS 16+, macOS 13+) consumed by OralableApp.
No external SPM dependencies — lightweight shared library.

2. BLE STACK LAYERS (top to bottom)
-----------------------------------

+-------------------------------------------------------------------------+
|  UI / ViewModels (DashboardViewModel, etc.)                             |
`-------------------------------+-----------------------------------------+
                                | Combine publishers
+-------------------------------v-----------------------------------------+
|  DeviceManager + coordinators                                           |
|  - DeviceConnectionCoordinator  (connect, discover, enable notify)      |
|  - DeviceScanningCoordinator    (scan for TGM service UUID)             |
|  - DeviceSensorDataRouter       (route batches to storage/UI)           |
|  - BLEBackgroundWorker          (reconnect, RSSI, health checks)      |
|  - BLEDataPublisher             (@Published UI state)                   |
|  - SensorDataProcessor          (HR, SpO2, history, auto-flush)       |
`-------------------------------+-----------------------------------------+
                                |
+-------------------------------v-----------------------------------------+
|  OralableDevice (BLEDeviceProtocol)                                     |
|  - CBPeripheralDelegate                                                 |
|  - Service/characteristic discovery                                     |
|  - enableNotifications / enableAccelerometerNotifications / temp      |
|  - OralableDevice+DataParsing -> BLEDataParser (OralableCore)          |
`-------------------------------+-----------------------------------------+
                                |
+-------------------------------v-----------------------------------------+
|  BLECentralManager (implements BLEService protocol)                     |
|  - CBCentralManager delegate                                            |
|  - connect / disconnect / scan / setNotifyValue                         |
`-------------------------------+-----------------------------------------+
                                |
+-------------------------------v-----------------------------------------+
|  CoreBluetooth (iOS)  <--BLE-->  nRF52832 firmware (tgm_service)        |
`-------------------------------------------------------------------------+

3. CONNECTION FLOW (DeviceConnectionCoordinator)
------------------------------------------------
Step 1: handleDeviceConnected -> readiness = .connected
Step 2: discoverServices([3A0FF000, 180F]) — 10s timeout
Step 3: discoverCharacteristics — 10s timeout
```

```
Step 2.5: requestFirmwareConnParamUpdate() (OralableDevice)
Step 3: readFirmwareVersion() from 3A0FF006 — FirmwareGate blocks old FW
Step 4: enableNotifications() on 3A0FF001 (PPG) — 10s timeout
Step 5: enableAccelerometerNotifications() on 3A0FF002
Step 5b: enableTemperatureNotifications() on 3A0FF003
Step 5c: configurePPGLEDs() (register writes via 3A0FF008)
-> readiness = .ready
-> automaticRecordingSession?.onDeviceConnected()

NOT enabled by iOS app today:
  - 3A0FF007 (PPG reg read notify)
  - 3A0FF008 (PPG reg write — used only for LED config)
  - 3A0FF009 (Status: worn/charging/battery_pct) — firmware has it, iOS does not subscribe

iOS also references (pcb00003 may not expose):
  - 3A0FF00A firmware log notify
  - 3A0FF00B firmware config write
  - 3A0FF00C firmware config state

4. NOTIFICATION READINESS (OralableDevice)
------------------------------------------
struct NotificationReadiness: OptionSet {
  ppgData, accelerometer, temperature, battery
  allRequired = [.ppgData, .accelerometer]
  all = [.ppgData, .accelerometer, .temperature, .battery]
}

Device is "ready for streaming" when PPG + ACC CCC confirmations received.
Battery CCC enabled separately (deferred path).

5. DATA PARSING PIPELINE
------------------------
BLE notify (3A0FF001)
  -> OralableDevice.parseSensorData(Data)
  -> OralableCore.BLEDataParser.parsePPGPacket()
  -> [SensorReading] batch (Red, IR, Green per sample)
  -> readingsBatchSubject (Combine)
  -> DeviceManager.handleSensorReadingsBatch()
  -> readingsBatchSubject / latestReadings / allSensorReadings
  -> SensorDataProcessor / DashboardViewModel / AutomaticRecordingSession

Accelerometer (3A0FF002):
  -> BLEDataParser.parseAccelerometerPacket()
  -> SensorReading batches (X, Y, Z)

Temperature (3A0FF003):
  -> parseTemperaturePacket() -> TemperatureData

Battery (3A0FF004):
  -> int32 mV -> BatteryConversion -> percentage

PPG packet expectations (OralableCore.BLEConstants.TGM):
  - PPG: 244 bytes = 4-byte frame counter + 20 samples × 12 bytes
  - ACC: 154 bytes = 4-byte frame counter + 25 samples × 6 bytes
  - Temp: 8 bytes
  - Battery: 4 bytes (int32 mV)
```

```
6. ORALABLECORE BLE MODULE
--------------------------
Sources/OralableCore/BLE/
  BLEConstants.swift   — All TGM UUIDs, packet sizes, RSSI thresholds
  BLEDataParser.swift  — Byte-level parsing, timestamp interpolation
  SensorDataBuffer.swift — Circular buffering for real-time streams

Sources/OralableCore/Algorithms/
  ButterworthFilter.swift, PPGProcessor.swift, IRDCProcessor.swift
  TransferFunctionFilter.swift — IIR filters matching Python spec

Sources/OralableCore/Calculations/
  BiometricProcessor, HeartRateService, SpO2Service
  MotionCompensator, ActivityClassifier
  MAMInferenceManager — Core ML bruxism inference (BruxismMAM_Temporalis.mlpackage)

Sources/OralableCore/Events/
  AutomaticRecordingSession — auto-start recording on connect
  EventDetector, StateTransitionDetector
  EventRecordingSession

7. SENSOR DATA PROCESSING (SensorDataProcessor)
-----------------------------------------------
- Maintains sensorDataHistory (max 10,000 rows)
- flushLiveHistoryToTempFileIfNonEmpty() — hourly spill to Application Support
- BioMetricCalculator: peak detection HR, R-ratio SpO2
- Calibration capture window for Temporalis wizard (10 min, ~40k samples)

8. BACKGROUND / RECONNECTION (BLEBackgroundWorker)
--------------------------------------------------
Default config:
  maxReconnectionAttempts: 3  <- LIMITS 8+ hour sessions
  baseReconnectionDelay: 2s (exponential backoff, max 30s)
  rssiPollingInterval: 5s
  healthCheckInterval: 10s
  connectionStaleTimeout: 30s
  autoReconnectEnabled: true

Gap for long sessions: only 3 reconnect attempts then gives up.
AutomaticRecordingSession ends on disconnect (no pause/resume across gaps).

9. SCANNING
-----------
DeviceScanningCoordinator filters for TGM service UUID:
  3A0FF000-98C4-46B2-94AF-1AEE0FD4C48E

BLECentralManager also checks advertised service UUIDs for Oralable detection.

10. FIRMWARE <-> iOS INTERACTION (1.0.36)
-----------------------------------------
Firmware worn-gating:
  - Off-body: PPG/ACC/temp notifies suppressed even with CCC enabled
  - On-body: streams resume if CCC still on

iOS today:
```

```
  - Does NOT subscribe to 009 Status for firmware `worn` bit
  - Infers worn from PPG perfusion / signal quality in app logic
  - Enables PPG+ACC+temp+battery on connect regardless of worn state
  - With 1.0.36: off-body = minimal BLE from device side automatically

Recommended iOS changes for 8+ hour adaptive sessions:
  1. Subscribe to 3A0FF009 Status, parse worn byte
  2. Unlimited/indefinite reconnect during active recording
  3. Session pause/resume across brief disconnects (not fresh session)
  4. More aggressive AutoFlush (30 min or sample-count threshold)
  5. Gate UI "streaming" indicators on firmware worn bit

11. KEY SWIFT FILES
-------------------
OralableApp/OralableApp/Managers/
  DeviceManager.swift
  DeviceConnectionCoordinator.swift
  DeviceScanningCoordinator.swift
  DeviceSensorDataRouter.swift
  BLECentralManager.swift
  BLE/BLEDataPublisher.swift
  BLE/BLEBackgroundWorker.swift
  BLE/SensorDataProcessor.swift
  BLE/BioMetricCalculator.swift
  AutoFlushService.swift

OralableApp/OralableApp/Devices/OralableDevice/
  OralableDevice.swift
  OralableDevice+CBPeripheralDelegate.swift
  OralableDevice+DataParsing.swift

OralableCore/Sources/OralableCore/
  BLE/BLEConstants.swift, BLE/BLEDataParser.swift
  Events/AutomaticRecordingSession.swift
  Calculations/MAMInferenceManager.swift

12. DATA FLOW DIAGRAM
---------------------
  nRF52832                    iOS
  --------                    ---
  PPG sample @50Hz
       |
  batch 20 samples
       |
  GATT notify 001 ----------> CBPeripheralDelegate
                                    |
                              parseSensorData()
                                    |
                              BLEDataParser (OralableCore)
                                    |
                              SensorReading batch
                                    |
                              DeviceManager router
                                    |
                    +---------------+---------------+
                    v               v               v
```

```
            SensorDataProcessor  Recording   Dashboard UI
                    |
            BiometricProcessor / MAMInferenceManager
                    |
            StateTransitionDetector -> events CSV

================================================================================
END iOS BLE STREAMING SUMMARY
================================================================================
```

4 Part IV — GitHub Repositories Overview

```
================================================================================
ORALABLE — FOUR GITHUB REPOSITORIES OVERVIEW
Generated: 2026-06-07
Owner: johnacogan67
================================================================================

This document consolidates metadata and README content from all four Oralable
codebases used in the MAM (Masseter / cheek PPG + accelerometer) platform.

================================================================================
REPO 1: cursor_oralable
================================================================================
URL:         https://github.com/johnacogan67/cursor_oralable
Description: (none set on GitHub)
Branch:      main
Commits:     13
Tracked files: 65
Primary languages: Python (.py), Markdown (.md)

PURPOSE
-------
Python research and signal-processing pipeline for Oralable MAM data.
Converts nRF Connect BLE logs to 50 Hz CSV, runs Temporalis gold validation,
clinical reports, and algorithm development that feeds iOS via OralableCore.

RECENT COMMITS
--------------
6dde116 Lower airway-rescue label threshold from 15% to 10%.
a89e315 Update temporalis validation outputs and regenerate 3D infographic assets.
6ced2ef Add OMG signature figure and plotting script for Figure 04.
c94474d Temporalis MAM: train from 50 Hz, remove masseter stub, pipeline script
14169c4 feat: Enhance Temporalis MAM model generation and processing

README (copy-pasted from GitHub)
--------------------------------
# Oralable MAM Signal Processing

## Setup (pandas)

A virtual environment with pandas is in `.venv`. To use it:

```bash
source .venv/bin/activate   # or on Windows: .venv\Scripts\activate
# Or run directly:
.venv/bin/python src/parser/log_parser.py "data/raw/text.txt"
```

To create the venv and install dependencies (if needed):

```
python3 -m venv .venv
.venv/bin/pip install --trusted-host pypi.org --trusted-host files.pythonhosted.org -r requireme
```

4.1 Data Flow

1. Place nrfConnect logs in `data/raw/`.
```

2. Run src/parser/log_parser.py to convert HEX to initial CSV.
3. Run src/processing/resampler.py to normalize to 50Hz.

4.2 Researcher Instructions (Temporalis gold chain)

Use a Python 3 environment with dependencies from requirements.txt (see **Setup** above).

1. **Record** a session following docs/TEMPORALIS_COLLECTION_PROTOCOL.md and save the BLE log (e.g. TEMPORALIS_RAW_01.csv or under data/raw/).
2. **Build the gold validation CSV** (50 Hz, Butterworth filters, automatic Temporalis labels, SpO2 / SASHB):

   ```
   python scripts/process_temporalis_gold.py path/to/your_ble_log.csv
   ```

   Output: data/validation/GOLD_STANDARD_VALIDATION.csv, plots under plots/temporalis_validation/.
3. **Optional — protocol segment table only** (no recording):

   ```
   python -m src.analysis.label_generator --temporalis-auto
   ```
4. **Clinical report** (TFI, SASHB, event counts, SpO2–clench timing) for IEEE / patent summaries:

   ```
   python scripts/generate_clinical_report.py --input data/validation/GOLD_STANDARD_VALIDATION.
   ```

   Console output and data/validation/clinical_report.txt (override with –-out).

Note: src/analysis/features.py defines calculate_tfi() (Temporalis Fatigue Index) used by the report script.

4.3 KEY DIRECTORIES

src/parser/ BLE log -> CSV (TDM parsing) src/processing/ 50 Hz resampler, filters src/analysis/ features.py, label_generator, clinical metrics src/utils/ sync_align.py (3-tap accel sync), ble_logger.py scripts/ process_temporalis_gold.py, generate_clinical_report.py docs/ ALGORITHM_ARCHITECTURE.md, IR_DC_ADC_FORMAT.md, protocols data/raw/ nRF Connect exports data/validation/ Gold standard outputs

4.4 SIGNAL PROCESSING RULES (from .cursorrules)

- All final data resampled to exactly 50 Hz (20 ms intervals)
- PPG: Butterworth bandpass 0.5–8.0 Hz (HR); low-pass <1 Hz (IR DC / occlusion)
- Accelerometer: actigraphy + jaw vibration; sync with 50 Hz PPG
- PPG channel order: R_G_IR (Red, Green, IR)
- IR-DC coupling range: 10M–70M raw (32-bit firmware)
- Clench detection cross-verified against IR DC-trough depth

==========

REPO 2: oralable_nrf ==========

URL: https://github.com/johnacogan67/oralable_nrf Description: (none set on GitHub) Branch: known-good-battery-ble (local) Commits: 4 (app layer; full workspace includes NCS/Zephyr submodules) Tracked files: 2601 (includes Zephyr/NCS build tree)

4.5 PURPOSE

nRF Connect SDK firmware for Oralable MAM device (nRF52832, pcb00003). Custom TGM GATT service streams PPG, accelerometer, temperature, battery. Firmware 1.0.36 adds worn-gated streaming for adaptive BLE bandwidth.

4.6 RECENT COMMITS

ddbc2d2 Fix app build module resolution and sensor compile path deb26b5 Harden iOS stream startup defaults and bump app version. c617f81 Stabilize battery-only runtime by hardening BATEN latch handling. de499b0 Flattened directory structure and moved iOS app out

4.7 README (copy-pasted from GitHub — excerpt)

5 tgm_firmware

Firmware for the TGM

6 nRF Connect SDK example application

Based on ncs-example-application. Application in `app/`.

6.0.1 Streaming PPG data

50 Hz, CONFIG_PPG_SAMPLES_PER_FRAME=20 (0.4 s per frame). Structure tgm_service_ppg_data_t: frame counter + samples (R4+IR4+G4 each).

6.0.2 Streaming accelerometer data

50 Hz, CONFIG_ACC_SAMPLES_PER_FRAME=25 (0.5 s per frame). Structure tgm_service_acc_data_t: frame counter + samples (X2+Y2+Z2 each).

6.0.3 Battery

int32 mV, CONFIG_BATTERY_MEASUREMENT_INTERVAL=300 s default.

6.0.4 Temperature

8 bytes, centidegree Celsius, CONFIG_TEMPERATURE_MEASUREMENT_INTERVAL=1 s.

6.0.5 PPG register tuning

Write 2 bytes to 3a0ff008: register address + value.

6.1 KEY DIRECTORIES

app/src/ Application (main, ble, tgm_service, ppg, acc, battery) app/prj.conf Kconfig (BLE buffers, sample rates, supervision timeout) boards/byteexplain/pcb00003/ Board definition scripts/ flash_and_rtt.sh docs/ TANDEM_WORKFLOW.md, compatibility_matrix.md

CURRENT VERSION: 1.0.36-nrfconnect (app/VERSION)

==========

REPO 3: oralable_swift ==========

URL: https://github.com/johnacogan67/oralable_swift Description: (none set on GitHub) Branch: main Commits: 398 Tracked files: 433 Primary languages: Swift (261 files), JSON, HTML

6.2 PURPOSE

iOS application (OralableApp) for consumer and professional (dentist) targets. BLE central, device pairing, real-time dashboard, recording sessions, CloudKit, subscriptions, clinical exports. Depends on OralableCore Swift package.

6.3 RECENT COMMITS

547c2b7 Document FirstLaunch onboarding fit-guide race (readiness vs pairing) ff0e4d9 Fix SharedDataManager skip-sync log string interpolation a6a2bf1 Improve BLE resilience and add firmware controls ad9b062 Add BLE link-quality RSSI path and debug summaries b7adf56 Improve BLE disconnect handling for supervision timeouts

README: (no root README on GitHub)

6.4 KEY DIRECTORIES

OralableApp/OralableApp/ Managers/ DeviceManager, BLE coordinators, background worker Devices/ OralableDevice, ANRMuscleSenseDevice Services/ SignalProcessingPipeline, UnifiedBiometricProcessor Views/ Dashboard, historical, settings, onboarding Models/ Sensor models, recording sessions OralableApp/OralableApp.xcodeproj/ .github/workflows/ ios.yml CI

6.5 TARGETS

- OralableApp (consumer)
- OralableForProfessionals (dentist/clinical)

6.6 PLATFORM

iOS, CoreBluetooth, CloudKit, StoreKit subscriptions

==========

REPO 4: OralableCore ==========

URL: https://github.com/johnacogan67/OralableCore Description: (none set on GitHub) Branch: main Commits: 60 Tracked files: 127 Primary languages: Swift (122 files), Core ML model

6.7 PURPOSE

Shared Swift package: BLE parsing, signal processing algorithms, biometric calculations, event detection, CSV export, CloudKit models, Core ML inference. Single source of truth for protocol constants and parsing logic used by iOS app.

6.8 RECENT COMMITS

9de931a Use stream timestamps for state events and harden inference gaps. 3c8803b Suppress MAM gap/OOD warnings during warm-up 9ad0bfc Throttle MAM diagnostic debug logs during live inference. 0c74cf2 Throttle MAM input-gap warning logging to real gaps. cdc53be Use absolute IR-DC shift magnitude for activity gate decisions.

README: (no README on GitHub)

6.9 PACKAGE STRUCTURE (Package.swift)

name: OralableCore platforms: iOS 16+, macOS 13+ products: .library(name: “OralableCore”) dependencies: none (lightweight)

Sources/OralableCore/ BLE/ BLEConstants, BLEDataParser, SensorDataBuffer Algorithms/ ButterworthFilter, PPGProcessor, IRDCProcessor Calculations/ HR, SpO2, BiometricProcessor, MAMInferenceManager

Filters/ TransferFunctionFilter Events/ AutomaticRecordingSession, EventDetector, StateTransitionDetector CSV/ Parser, Exporter, StateEventCSVExporter CloudKit/ HealthDataRecord, SharedSessionData Models/ SensorData, SensorReading, PPGData, DeviceInfo Resources/ BruxismMAM_Temporalis.mlpackage (Core ML) DesignSystem/ Colors, typography, shared UI components

Tests/OralableCoreTests/ ParityTests, BLETests, CalculationsTests, IntegrationTests Resources/GOLD_STANDARD_FILTER_

==========
CROSS-REPO RELATIONSHIPS ==========

cursor_oralable –algorithm spec / training–> OralableCore (Core ML models) l l l BLE log analysis l Swift Package v v validation CSV oralable_swift (iOS app) l l BLE GATT v oralable_nrf (firmware)

Data path (production): Device (nRF) -> BLE notify -> iOS OralableDevice -> OralableCore.BLEDataParser -> MAMInferenceManager -> events / dashboard / CloudKit / CSV export

Data path (research): nRF Connect CSV -> cursor_oralable log_parser -> 50 Hz CSV -> features / gold standard -> train Core ML -> OralableCore Resources

========== END
GITHUB REPOS OVERVIEW ==========

\newpage

Part V — Algorithm & System Integration

==========
ORALABLE MAM — ALGORITHM & SYSTEM INTEGRATION Generated: 2026-06-07 Sources: cursor_oralable/docs/ALGORITHM_ARCHITECTURE.md + live codebase ==========

6.10 1. PRODUCT CONTEXT

Device: Oralable MAM (PPG Red/IR/Green + 3-axis accelerometer) Location: Cheek (temporalis / masseter region) Goal: Detect sleep bruxism (clenching/grinding) via hemodynamic occlusion Vitals: HR (Green), SpO2 (Red/IR), HRV for sleep staging

6.11 2. SAMPLING & SYNC STANDARDS

- Final data: exactly 50 Hz (20 ms) via linear interpolation
- PPG bandpass: Butterworth 0.5–8.0 Hz (heart rate)
- IR DC: low-pass <1 Hz (muscle occlusion / clench indicator)
- Accelerometer: up to 100 Hz native; synced/resampled to 50 Hz with PPG
- Sync anchor: 3 consecutive high-G spikes on accel Z (“sync taps”)

6.12 3. PYTHON MODULES (cursor_oralable)

src/analysis/features.py - Butterworth filters, beat detection, IR DC baseline - 5s window biomarkers: ir_dc_shift, rise_fall_symmetry, HRV SVD

src/analysis/visualize_test.py - Bandpass 0.5–8 Hz, median filter accel, rolling mean IR DC

src/utils/sync_align.py - 3-tap sync detection on accel Z (2s window, 3σ threshold)

src/parser/log_parser.py - TDM parsing from nRF Connect HEX exports, 50 Hz resampling

src/processing/resampler.py - 50 Hz linear interpolation

6.13 4. SWIFT MODULES (OralableCore + oralable_swift)

OralableCore/Algorithms/ ButterworthFilter.swift vDSP IIR PPGProcessor.swift Bandpass + beat detection IRDCProcessor.swift Lowpass + rolling mean for occlusion

OralableCore/Calculations/ HeartRateService, SpO2Service, BiometricProcessor MotionCompensator, ActivityClassifier MAMInferenceManager Core ML bruxism classifier

oralable_swift/Services/ UnifiedBiometricProcessor HR, SpO2, motion compensation, perfusion SignalProcessingPipeline Activity classification pipeline

6.14 5. ARCHITECTURE: SINGLE SOURCE OF TRUTH (target state)

+————————————————————————+ | ALGORITHM SPEC (YAML/JSON) | | - Filter params (0.5–8 Hz, <1 Hz, order 4) | | - Window sizes (50 samples, 5s, 100 samples) | | - Sync tap params (2s window, 3σ, min 80ms between taps) | | - SpO2 calibration coefficients | ————————————————————————

```
----------------------+          |                                  |          v
v +---------------------+         +-----------------------------+ |  Python (Research)
|             |  Swift (Production)          | |  features.py         |            |
OralableCore/Algorithms     | |  scipy.signal        |             |  Accelerate vDSP
| |  Core ML export      |            |  Core ML inference          |————————+
‘——————————+
```

Implemented in OralableCore: Signal/AlgorithmSpec.swift — shared filter parameters Filters/TransferFunctionFilter.swift — IIR coefficients Resources/BruxismMAM_Temporalis.mlpackage — trained from Python pipeline

6.15 6. ALGORITHM SPEC PARAMETERS (reference)

sampling: ppg_hz: 50.0 accel_hz: 100.0

filters: ppg_bandpass: 0.5–8.0 Hz, order 4 ir_dc_lowpass: 0.8 Hz, order 4 accel_median: window 5

buffers: ppg_circular_size: 100 (2s at 50 Hz) hr_window_samples: 150 (3s) spo2_window_samples: 150

sync_taps: window_seconds: 2.0 sigma_threshold: 3.0 min_distance_ms: 80

spo2_calibration (empirical): $SpO2 = -45.060 R^2 + 30.354 R + 94.845$

6.16 7. CLENCH / BRUXISM DETECTION LOGIC

Primary signal: IR DC-trough depth (hemodynamic occlusion during clench) Cross-verification: accelerometer jaw vibration + PPG morphology ML path: MAMInferenceManager + BruxismMAM_Temporalis Core ML model - Input: windowed biomarkers from 50 Hz stream - Output: bruxism / muscle activity classification

StateTransitionDetector: - Uses stream timestamps for event alignment - Activity gate uses absolute IR-DC shift magnitude

6.17 8. IR-DC ADC FORMAT (firmware -> app)

See docs/IR_DC_ADC_FORMAT.md in cursor_oralable - 32-bit raw IR values on pcb00003 - IR-DC coupling range: 10M–70M raw for cheek coupling - Channel order: R_G_IR when BLE slot1 is constant

6.18 9. VALIDATION STRATEGY

1. Python gold chain: process_temporalis_gold.py -> GOLD_STANDARD_VALIDATION.csv
2. OralableCore ParityTests vs GOLD_STANDARD_FILTER_PARITY.csv
3. nRF Connect session logs (Oralable (7)–(11)) for BLE stability
4. Cross-verify clench vs IR DC-trough depth per project rules

5.

6.19 KNOWN GAPS (as of 2026-06-07)

Firmware:

- Battery keepalive visible as only 1 packet in some nRF Connect exports
- iOS does not yet use 009 Status characteristic

iOS long-session: - BLEBackgroundWorker: max 3 reconnect attempts - AutomaticRecordingSession: ends on disconnect (no pause/resume) - AutoFlush: ~hourly; unified buffer 1.8M samples

Algorithm parity: - Swift SpO2 still uses empirical curve; Python has fuller PWA pipeline - Ongoing alignment via AlgorithmSpec + parity tests

6.20 11. REFERENCES

- Peter Charlton Pulse Waveform Analysis (arterial stiffness, HRV)
- Zhang et al. 2023 PPG-Net 4 morphology (laminar/stagnant/turbulent)
- Attia et al. 2024 — 50 Hz sleep staging accuracy standards
- Apple Accelerate vDSP, Core ML Tools

========== END ALGORITHM & SYSTEM INTEGRATION ==========

‘‘‘